

UNIVERSIDAD CATÓLICA SANTO TORIBIO DE MOGROVEJO
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN



**IMPLEMENTACIÓN DE UNA API CON BASE EN REDES NEURONALES
CONVOLUCIONALES PARA LA VALIDACIÓN E IDENTIFICACIÓN DE
ESPECIES DE ÁRBOLES EN CHICLAYO**

**DOCUMENTO DE AVANCE DE TESIS
PARA EVIDENCIAR LA EJECUCIÓN DE TESIS
EN EL CURSO DE SEMINARIO DE TESIS I**

AUTOR

CINTHIA NAYELI SÁNCHEZ CIEZA

ASESOR

Mgtr. HUILDER JUANITO MERA MONTENEGRO

<https://orcid.org/0000-0001-6830-5415>

Chiclayo, 2024

INFORMACIÓN GENERAL

1. Facultad y Escuela

Facultad de Ingeniería – Escuela de Ingeniería de Sistemas y Computación

2. Título del informe de tesis

Implementación de una api con base en redes neuronales convolucionales para la validación e identificación de especies de árboles en Chiclayo

3. Autor(es) y firma(s)

Cinthia Nayeli Sánchez Cieza

Autor

4. Asesor y firma

Mgtr. Huilder Juanito Mera Montenegro

Asesor

5. Línea y área de la investigación

Desarrollo e innovación tecnológica - Inteligencia artificial

6. Fecha de presentación

28 de noviembre de 2024

ÍNDICE

I.	RESULTADOS.....	4
1.1.	EN BASE A LA METODOLOGÍA UTILIZADA	4
1.1.1.	ITERACIÓN #1: COMPRENSIÓN DEL NEGOCIO	4
1.1.2.	ITERACIÓN #2: COMPRENSIÓN DE LOS DATOS	10
1.1.3.	ITERACIÓN #3: PREPARACIÓN DE LOS DATOS.....	20
1.1.4.	ITERACIÓN #4: MODELADO.....	21
1.1.5.	ITERACIÓN #4: EVALUACIÓN	21
1.1.6.	ITERACIÓN #4: DESPLIEGUE	21
1.2.	EN BASE A LOS OBJETIVOS DEL PROYECTO.....	39
1.2.1.	GENERAR DATASETS DE IMÁGENES, INCLUYENDO UN DATASET CON IMÁGENES DE DIVERSAS ESCENAS Y OTRO CON IMÁGENES Y CARACTERÍSTICAS DE ÁRBOLES NATIVOS DE CHICLAYO.....	40
1.2.2.	IMPLEMENTAR UN MODELO DE APRENDIZAJE AUTOMÁTICO PARA VALIDAR A PRESENCIA DE ÁRBOLES EN UNA IMAGEN.....	41
1.2.3.	IMPLEMENTAR UN MODELO DE APRENDIZAJE AUTOMÁTICO PARA IDENTIFICAR DIFERENTES ESPECIES DE ÁRBOLES A PARTIR DE IMÁGENES.....	41
1.2.4.	DESARROLLAR UNA API REST QUE INTEGRE LOS MODELOS DE VALIDACIÓN E IDENTIFICACIÓN.....	41
1.2.5.	IMPLEMENTAR UN MODELO DE APRENDIZAJE AUTOMÁTICO PARA IDENTIFICAR DIFERENTES ESPECIES DE ÁRBOLES A PARTIR DE IMÁGENES.....	42

I. RESULTADOS

1.1. En base a la metodología utilizada

1.1.1. Iteración #1: Comprensión del negocio

1.1.1.1. Determinar los objetivos del negocio:

Objetivos principales:

- Mejorar la gestión y conservación del arbolado urbano en Chiclayo.
- Facilitar la identificación precisa y rápida de especies de árboles para autoridades municipales, investigadores y ciudadanos.
- Contribuir a la planificación urbana sostenible y al aumento de áreas verdes en la ciudad.
- Apoyar iniciativas de educación ambiental y concientización sobre la importancia de los árboles urbanos.
- Prevenir la tala indiscriminada de árboles en proyectos de remodelación urbana.

Criterios de éxito:

- Aumentar la precisión en la identificación de especies arbóreas en Chiclayo.
- Reducir el tiempo necesario para identificar y validar especies de árboles.
- Incrementar el número de árboles correctamente identificados y registrados en la ciudad.
- Disminuir los casos de tala injustificada de árboles en proyectos urbanos.

1.1.1.2. Evaluar la situación actual:

Análisis de la realidad estudiada:

El estudio y reconocimiento de las especies arbóreas en áreas urbanas se han vuelto esenciales para avanzar en la

gestión de recursos y mejorar la calidad medioambiental de las ciudades. En Chiclayo, la gestión del arbolado urbano enfrenta desafíos significativos debido a la falta de herramientas tecnológicas eficientes para la identificación y validación de especies arbóreas. Los métodos actuales son principalmente manuales, lo que resulta en procesos lentos, costosos y propensos a errores.

Evaluación detallada de la realidad:

A nivel local, en la ciudad de Chiclayo, aunque la extensión de áreas verdes es de aproximadamente 1.229.760 m², el índice de áreas verdes por habitante es significativamente bajo. La falta de un sistema automatizado para la identificación de árboles ha llevado a decisiones inadecuadas en la gestión urbana. Por ejemplo, la poda agresiva indiscriminada de 42 árboles en la Avenida Balta parece haber sido resultado de la carencia de un inventario del arbolado urbano y el aparente desconocimiento por parte de la gestión municipal sobre el valor ecológico de los árboles maduros.



Fig. 1 Remoción de 42 árboles en avenida principal de Chiclayo sería inviable y afectaría salud de ciudadanos

Deficiencias identificadas:

- Ausencia de un inventario exhaustivo y actualizado de árboles en Chiclayo.
- Falta de herramientas tecnológicas accesibles para la validación e identificación de árboles.
- Dependencia excesiva de expertos para la identificación manual de especies, lo que ralentiza los procesos de gestión.
- Carencia de datos precisos sobre la distribución y estado de las especies arbóreas en la ciudad.
- Falta de evaluaciones técnicas previas a las intervenciones en el arbolado urbano.
- Ausencia de procesos participativos que involucren a la ciudadanía en la toma de decisiones sobre el arbolado urbano.

Necesidades para el producto acreditable:

El proyecto busca desarrollar una API REST basada en redes neuronales convolucionales que abordará las siguientes necesidades:

- Validación automática de la presencia de árboles en imágenes.
- Identificación precisa de especies de árboles nativos de Chiclayo.
- Generación de datasets de imágenes para entrenamiento y validación de los modelos.
- Integración de los modelos de validación e identificación en una API accesible.
- Creación de una interfaz intuitiva para la carga y análisis de imágenes de árboles.

Funciones del producto acreditable:

La API REST a desarrollar tendrá las siguientes funciones principales:

- Analizar imágenes para validar la presencia de árboles.
- Identificar y clasificar especies de árboles nativos de Chiclayo.
- Proporcionar información detallada sobre las especies identificadas.
- Permitir la administración y gestión de la base de datos de imágenes y modelos de aprendizaje automático.
- Facilitar la integración con otros sistemas de gestión urbana.

Recursos disponibles:

- Personal: Desarrollador del proyecto de tesis.
- Datos: Dataset de imágenes diversas, imágenes de árboles de Chiclayo (a ser recopiladas) para generar otro Dataset.
- Hardware: Computadora portátil.
- Software: Herramientas de desarrollo (Visual Studio Code, Android Studio, Postman), plataformas de aprendizaje automático (Google Colab).

Requerimientos funcionales:

1. Validar imágenes con árboles mediante ML:
 - Evita el procesamiento de imágenes irrelevantes
 - Ahorra tiempo y recursos computacionales
 - Asegura que solo se analicen imágenes pertinentes
2. Identificar especies de árboles:
 - Permite clasificación automática de especies
 - Facilita inventarios forestales
 - Ayuda en estudios de biodiversidad
3. Mostrar información detallada de especies:
 - Proporciona datos educativos y científicos
 - Ayuda en la toma de decisiones de conservación
 - Facilita el conocimiento de características específicas

4. Administración de base de datos y modelos:
 - Permite mantener actualizada la información
 - Facilita el mantenimiento de los modelos
 - Mejora la gestión y organización del sistema
 - Permite el seguimiento histórico de datos

Requerimientos no funcionales:

1. Interfaz intuitiva y amigable:
 - Reduce la curva de aprendizaje
 - Minimiza errores de usuario
 - Aumenta la satisfacción y adopción del sistema
2. Tiempos de respuesta rápidos:
 - Mejora la experiencia del usuario
 - Permite procesar más imágenes en menos tiempo
 - Facilita el trabajo en campo o situaciones urgentes
3. Alta precisión en identificación:
 - Garantiza resultados confiables
 - Reduce la necesidad de verificación manual
 - Aumenta la credibilidad del sistema
4. Escalabilidad:
 - Asegura el crecimiento futuro del sistema
 - Permite adaptarse a nuevas necesidades
 - Mantiene el rendimiento con más datos
5. Seguridad en datos:
 - Protege información sensible
 - Cumple con regulaciones de privacidad
 - Previene pérdidas o alteraciones de datos
6. Fácil integración:
 - Permite conexión con otros sistemas
 - Facilita la expansión de funcionalidades
 - Mejora la interoperabilidad
7. Documentación clara:
 - Facilita el mantenimiento
 - Ayuda en la capacitación de usuarios
 - Reduce dependencia de personal específico

8. Soporte y mantenimiento:

- Garantiza funcionamiento continuo
- Permite actualizaciones y mejoras
- Resuelve problemas rápidamente

PRESUPUESTO DE REQUISITOS

REQUISITOS FUNCIONALES (Enfocados en funcionalidad principal):

1. Laptop Core i5: S/. 3,352.00 (70% del costo) = S/. 2,346.40
 - Principal equipo para desarrollo de modelos ML
 - Procesamiento de imágenes
 - Desarrollo de funcionalidades core
2. VPS: S/. 285.00 (100%)
 - Alojamiento de modelos ML
 - Base de datos de imágenes
3. Google Colab: S/. 153.00 (100%)
 - Entrenamiento de modelos ML
 - Procesamiento de datos

Subtotal Funcionales: S/. 2,784.40

REQUISITOS NO FUNCIONALES (Enfocados en calidad y soporte):

1. Laptop Core i5: S/. 3,352.00 (30% del costo) = S/. 1,005.60
 - Pruebas de rendimiento
 - Optimización de interfaz
 - Seguridad y respaldos
2. Teléfono Android: S/. 600.00 (100%)
 - Pruebas de interfaz móvil
 - Verificación de usabilidad
3. Internet: S/. 90.00 (100%)
 - Conectividad para integraciones
 - Pruebas de rendimiento
 - Soporte y actualizaciones

Subtotal No Funcionales: S/. 1,695.60

RESUMEN:

- Presupuesto Requisitos Funcionales: S/. 2,784.40 (62%)
- Presupuesto Requisitos No Funcionales: S/. 1,695.60 (38%) = Total: S/. 4,480.00 (100%)

Supuestos y restricciones:

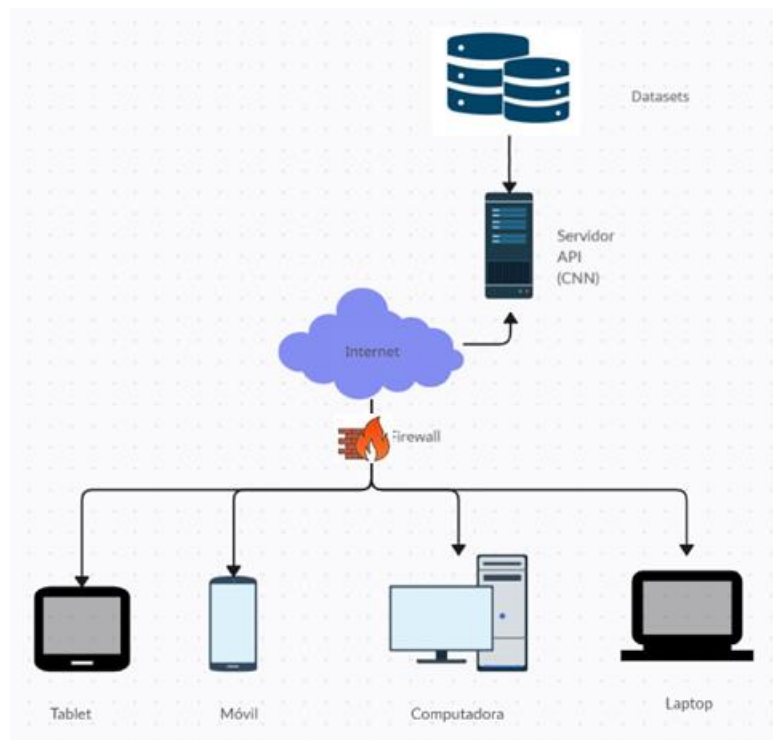
- Se asume la disponibilidad de suficientes imágenes de árboles de Chiclayo para el entrenamiento de los modelos.
- El proyecto debe ser de código abierto y uso gratuito.
- Se debe garantizar la privacidad y seguridad de los datos recopilados.

Riesgos y contingencias:

- Riesgo: Insuficientes datos de entrenamiento para algunas especies de árboles. Contingencia: Colaborar con expertos locales para recopilar más datos.
- Riesgo: Baja precisión inicial del modelo de identificación. Contingencia: Implementar un proceso iterativo de mejora y reentrenamiento del modelo.

Arquitectura:

La arquitectura que se considerará en la solución propuesta es la siguiente:



La infraestructura tecnológica estará conformada por:

✓ Dispositivos Clientes (Tablet, Móvil, Computadora, Laptop)

Son los dispositivos desde los cuales los usuarios interactúan con el sistema.

Pueden ser dispositivos móviles (tabletas, teléfonos inteligentes) o dispositivos de escritorio (computadoras de escritorio, laptops).

Estos dispositivos envían solicitudes a través de Internet para acceder a los servicios del sistema.

✓ Internet

Es la red global que permite la comunicación entre los dispositivos clientes y el servidor.

32

Proporciona la conectividad necesaria para que los dispositivos clientes puedan enviar solicitudes y recibir respuestas del servidor.

✓ Firewall

Es un componente de seguridad que actúa como barrera de protección entre la red interna y la red externa (Internet).

Filtra el tráfico entrante y saliente según las reglas de seguridad establecidas, permitiendo o bloqueando conexiones según sea

necesario.

Ayuda a proteger el sistema de amenazas externas, como ataques cibernéticos o accesos no autorizados.

✓ Servidor API (CNN)

Es el servidor que aloja la implementación de las Redes Neuronales Convolucionales (CNN).

Recibe las solicitudes de los dispositivos clientes a través de Internet y el Firewall.

Utiliza los modelos de CNN entrenados para procesar las imágenes recibidas y realizar la identificación de especies de

árboles.

Proporciona una interfaz de programación de aplicaciones (API)

para que los dispositivos clientes puedan interactuar con el sistema.

✓ Datasets

Son los conjuntos de datos utilizados para entrenar los modelos

de CNN.

Contienen imágenes de diferentes especies de árboles, etiquetadas con sus respectivas clasificaciones.

Los Datasets son utilizados durante la fase de entrenamiento de

los modelos de CNN para ajustar sus pesos y parámetros. Están

almacenados en servicio de almacenamiento en la nube.

Glosario de terminología:

- CNN: Redes Neuronales Convolucionales
- API REST: Interfaz de Programación de Aplicaciones basada en Transferencia de Estado Representacional
- Dataset: Conjunto de datos para entrenamiento y prueba de modelos

1.1.1.3. Establecer los objetivos de minería de datos:

Objetivos técnicos:

- Desarrollar un modelo de CNN capaz de validar la presencia de árboles en imágenes con alta precisión.
- Crear un modelo de CNN para identificar especies de árboles nativos de Chiclayo a partir de imágenes.

Criterios de éxito técnicos:

- Alcanzar una precisión mínima del 90% en la validación de presencia de árboles.

COMPONENTES ESPERADOS:

APLICACIÓN WEB:



Respuesta archivo no compatible:



Respuesta de no árbol:

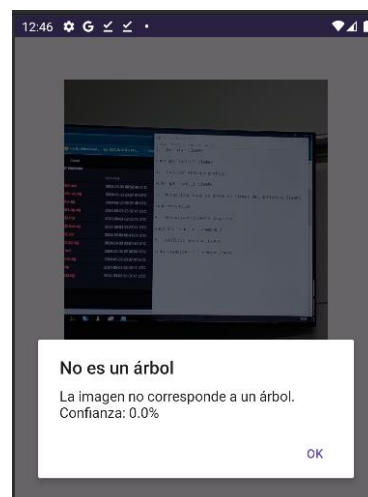


Respuesta de árbol:



APLICACIÓN MÓVIL:

Respuesta de no árbol:



Respuesta de árbol:



- Lograr una precisión mínima del 85% en la identificación de especies de árboles de Chiclayo.
- Obtener un tiempo de respuesta promedio de la API menor a 2 segundos por consulta.

1.1.1.4. Producir el plan del proyecto:

Item	Descripción / Fase	Duración (Días)	Comienzo	Fin	% Avance
01	Compresión del negocio	3	19/08/2024	21/08/2024	50%
02	Comprensión de los datos	4	22/08/2024	25/08/2024	
03	Preparación de los datos	4	26/08/2024	29/08/2024	
04	Modelado	7	30/08/2024	5/09/2024	
05	Evaluación del modelo implementado	15	6/09/2024	20/09/2024	
06	Documentación del 50%	14	21/09/2024	4/10/2024	
AVANCE AL 50%					
07	Despliegue del modelo implementado	2	5/09/2024	6/09/2024	100%
08	Diseño de la API REST	1	7/10/2024	8/10/2024	
09	Desarrollo de la API REST	1	9/10/2024	10/10/2024	
10	Pruebas de la API REST	2	11/10/2024	12/10/2024	
11	Diseño de la aplicación móvil	2	13/10/2024	14/10/2024	
12	Desarrollo de la aplicación móvil	2	15/10/2024	16/10/2024	
13	Pruebas de la aplicación móvil	2	17/10/2024	18/10/2024	
14	Documentación del 100%	21	19/10/2024	8/11/2024	
AVANCE AL 100%					100%
SUSTENTACION FINAL			15/11/2024	6/12/2024	

Fig. 2 Cronograma de actividades para la ejecución de la tesis

Duración estimada: 3 meses

Resultados esperados:

- API REST funcional para validación e identificación de árboles
- Modelos de CNN entrenados y validados
- Aplicaciones cliente (móvil)
- Documentación técnica y de usuario

Herramientas y técnicas a utilizar:

- Lenguaje de programación: Python
- Frameworks de aprendizaje profundo: TensorFlow
- Desarrollo de API: Flask
- Entorno de desarrollo: Visual Studio Code
- Plataforma de entrenamiento: Google Colab
- Pruebas de API: Postman

1.1.2. Iteración #2: Comprensión de los datos

a) Recopilación de datos:

Para este proyecto, se han obtenido dos conjuntos de datos principales:

- **Dataset de imágenes de diversas escenas:** Después de una exhaustiva investigación de datasets que contienen imágenes diversas, se decidió optar por el dataset CIFAR-100. Esta elección se basó en varios factores:

- Alta disponibilidad y fácil acceso para fines de investigación.
- Gran cantidad de imágenes y diversidad de clases.
- Variedad de escenas y objetos que permiten un entrenamiento robusto del modelo de validación de presencia de árboles.

Este dataset servirá como el conjunto de imágenes de diversas escenas requerido para el entrenamiento del modelo de validación de presencia de árboles.

- **Dataset de árboles de Chiclayo:** Para la creación de este dataset, se evaluaron diferentes opciones:

Inicialmente, se consideró capturar imágenes directamente de Google Street View y redimensionarlas manualmente. Sin embargo, este proceso resultó ser extremadamente laborioso y poco eficiente. Tras una investigación adicional, se optó por utilizar la API de Google Street View. Esta solución permite automatizar en gran medida el proceso de recopilación de imágenes mediante un script en Python.

El proceso implica:

- Hacer llamadas a la API de Google Street View a través de un código Python.
- Ingresar manualmente las coordenadas de latitud y longitud y otras características necesarias para la toma de las imágenes de árboles de interés en Chiclayo.

b) Descripción de los datos:

Dataset CIFAR-100:

- ✓ Número total de imágenes: 60,000
- ✓ Distribución: 50,000 imágenes para entrenamiento y 10,000 para pruebas
- ✓ Resolución de imágenes: 32x32 píxeles
- ✓ Formato de color: RGB (3 canales)
- ✓ Número de clases: 100
- ✓ Tamaño total del dataset: Aproximadamente 161 MB

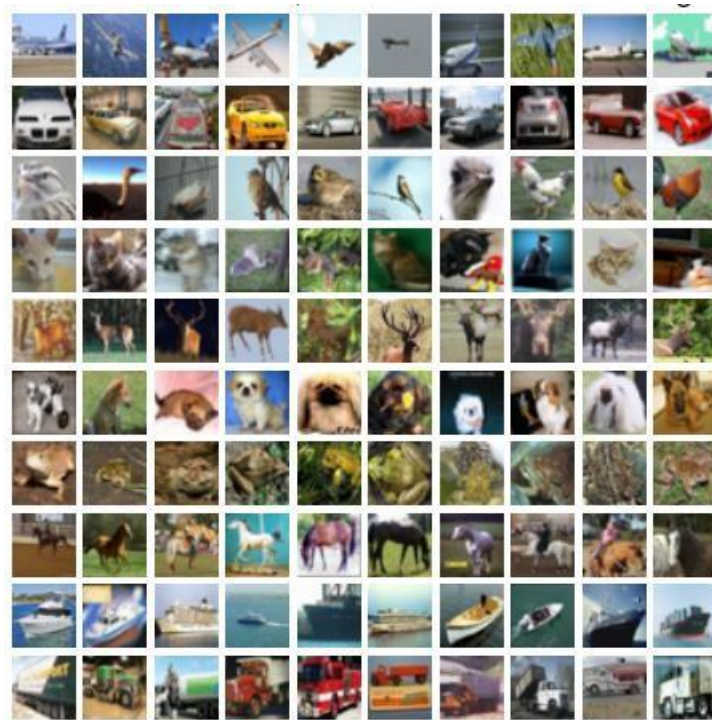


Fig. 3 Imagen referencial de Dataset Cifar-100

Dataset de árboles de Chiclayo generado por Google Street View

- ✓ Número total de imágenes: 500
- ✓ Distribución:
 - 70% para entrenamiento: 350 imágenes
 - 30% para pruebas: 150 imágenes
- ✓ Tamaño de imágenes: 640x480 píxeles
- ✓ Formato de color: RGB (3 canales)

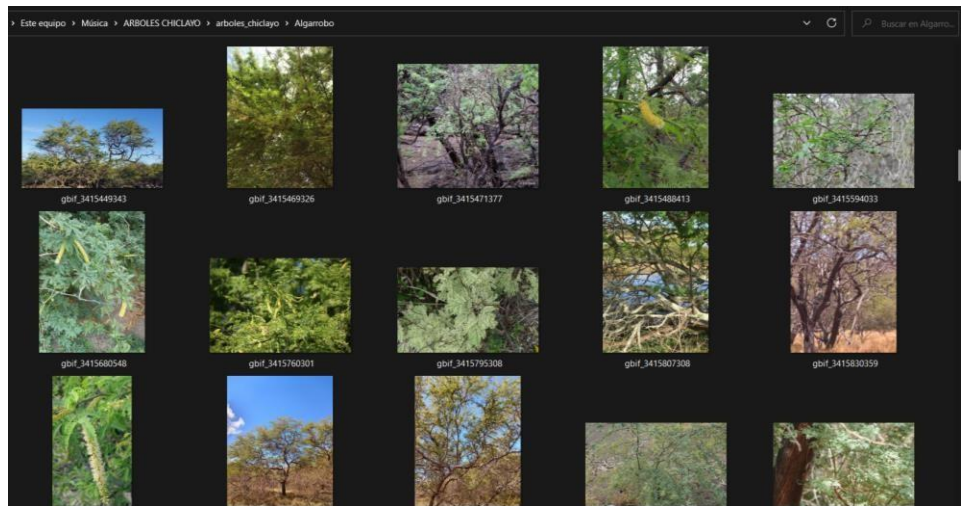
- ✓ Número de clases: 5
- ✓ Resolución de imágenes: 32x32 píxeles (redimensionadas para el entrenamiento)
- ✓ Tamaño total del dataset: Aproximadamente 40 MB



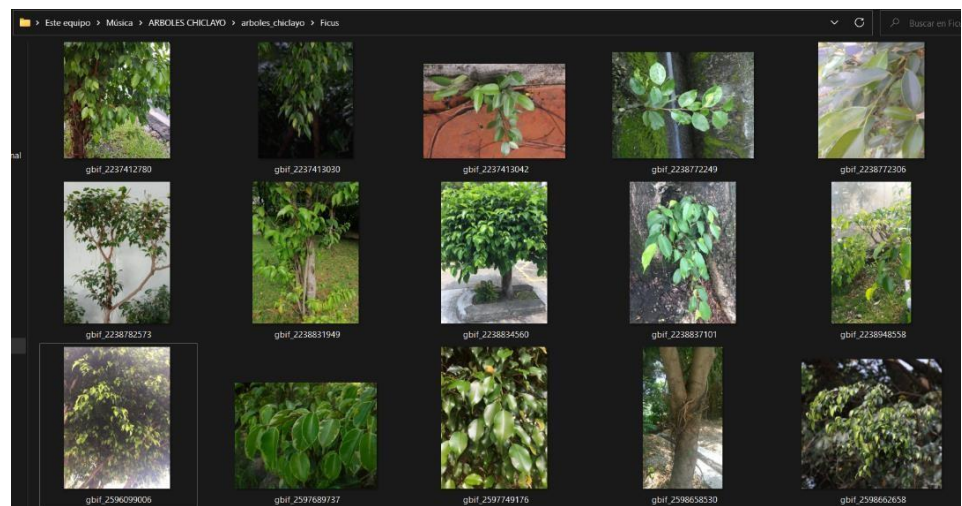
Fig. 3 Imagen referencial de árboles de Chiclayo

Asimismo, para tener aún más completo el dataset de imágenes de Chiclayo, se realizó una búsqueda de rutas de imágenes de cada especie a través de Internet, en la cual a través de un código se logró descargar cada imagen, a continuación, detallo el número de imágenes encontradas por cada especie:

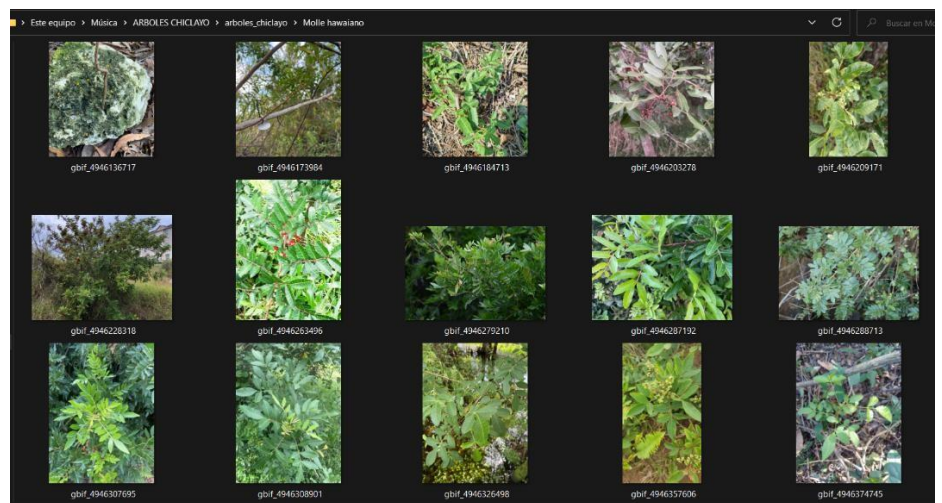
- Algarrobo: 507 imágenes



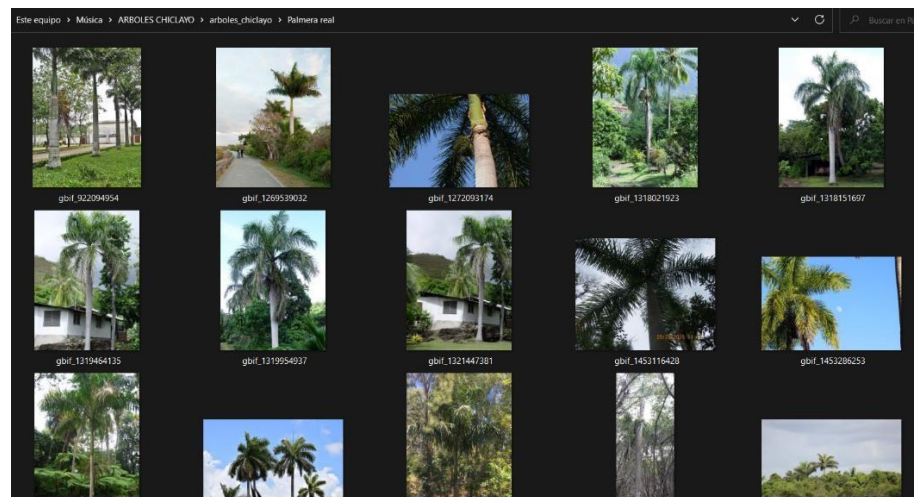
- Ficus: 505 imágenes



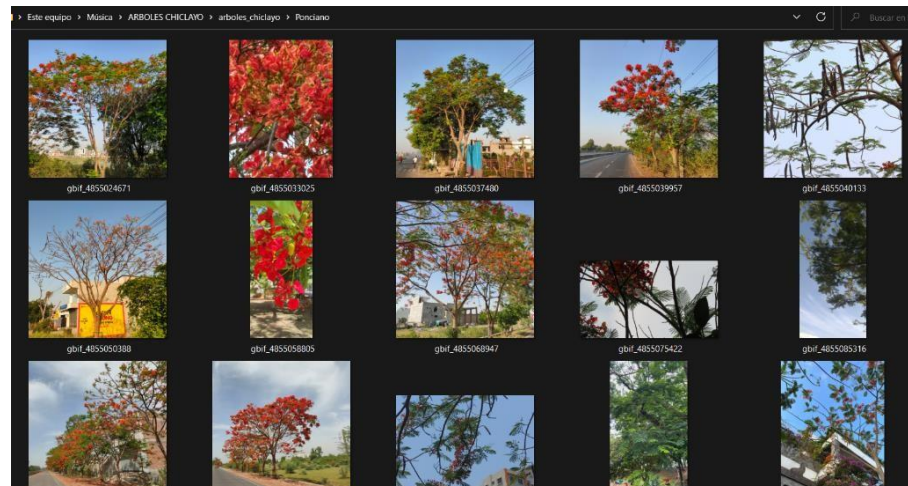
- Molle Hawaiano: 535 imágenes



- Palmera Real: 511 imágenes



- Ponciano: 510 imágenes



c) Exploración inicial de los datos:

Dataset CIFAR-100:

Se realizó un análisis exploratorio para identificar las clases con las que cuenta:

1. manzana
2. pez_de_acuario
3. bebé
4. oso
5. castor
6. cama

7. abeja
8. escarabajo
9. bicicleta
10. botella
11. tazón
12. niño
13. puente
14. autobús
15. mariposa
16. camello
17. lata
18. castillo
19. oruga
20. ganado
21. silla
22. chimpancé
23. reloj
24. nube
25. cucaracha
26. sofá
27. cangrejo
28. cocodrilo
29. taza
30. dinosaurio
31. delfín
32. elefante
33. pez_plano
34. bosque
35. zorro
36. niña
37. hámster
38. casa
39. canguro
40. teclado

41. lámpara
42. cortacésped
43. leopardo
44. león
45. lagarto
46. langosta
47. hombre
48. arce
49. motocicleta
50. montaña
51. ratón
52. hongo
53. roble
54. naranja
55. orquídea
56. nutria
57. palmera
58. pera
59. camioneta
60. pino
61. llanura
62. plato
63. amapola
64. puercoespín
65. zarigüeya
66. conejo
67. mapache
68. raya
69. carretera
70. cohete
71. rosa
72. mar
73. foca
74. tiburón

75. musaraña
76. mofeta
77. rascacielos
78. caracol
79. serpiente
80. araña
81. ardilla
82. tranvía
83. girasol
84. pimienta
85. mesa
86. tanque
87. teléfono
88. televisión
89. tigre
90. tractor
91. tren
92. trucha
93. tulipán
94. tortuga
95. armario
96. ballena
97. salsa
98. lobo
99. mujer
100. gusano

Asimismo, se visualizaron muestras aleatorias de imágenes para evaluar la calidad y variabilidad de las escenas representadas.

Dataset de árboles de Chiclayo

Se definió 5 clases, las cuales son 5 especies de árboles de Chiclayo:

1. algarrobo

2. delonix regia
3. ficus benamina
4. palma canaria
5. pimentero falso

d) Verificación de la calidad de los datos:

Dataset CIFAR-100:

Se verificó la integridad del dataset, asegurando que todas las imágenes estén presentes y sean legibles.

Se comprobó la consistencia en el tamaño y formato de las imágenes.

Se revisaron las etiquetas de las clases para asegurar su correcta asignación.

Dataset de árboles de Chiclayo

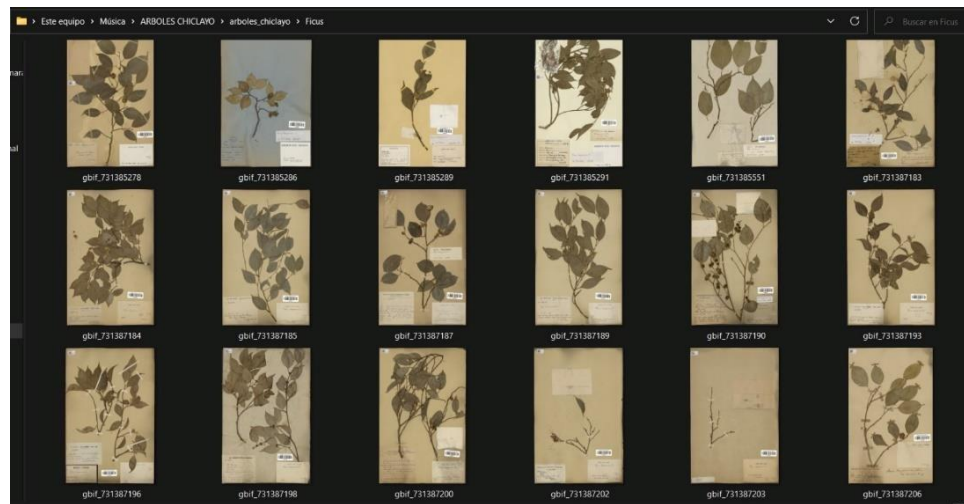
Se verificó la integridad del dataset, asegurando que todas las imágenes estén presentes y sean legibles.

Se comprobó la consistencia en el tamaño y formato de las imágenes.

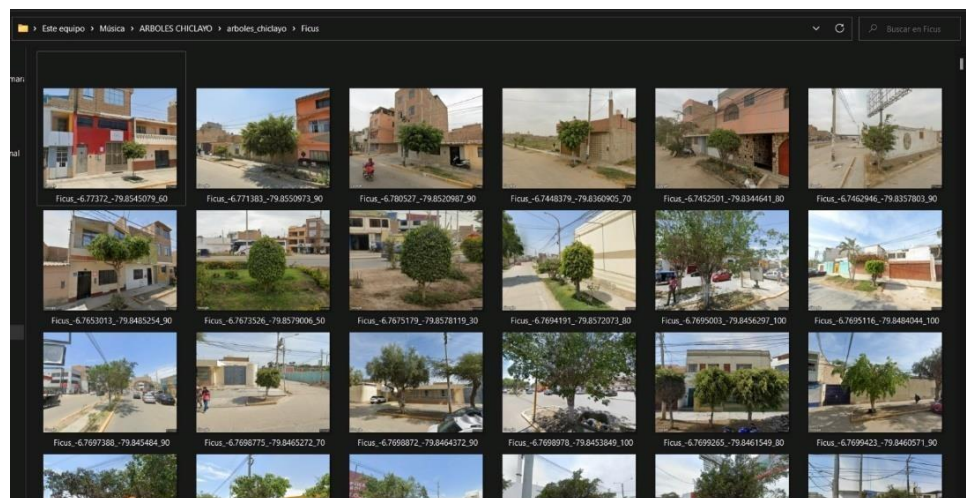
Se revisaron las etiquetas de las clases para asegurar su correcta asignación.

Finalmente se eliminaron imágenes que no son oportunas para el entrenamiento, como las siguientes imágenes:

Aquí por ejemplo son imágenes de hojas de esa especie, pero son hojas secas, cuestión que no es ayuda a la CNN



O aquí también podemos observar imágenes de árboles que están muy alejadas, la cual no permite a la CNN definir bien el árbol:



e) Identificación de problemas y desafíos:

Especies subrepresentadas: Se ha detectado la necesidad de aumentar el número de especies arbóreas en el dataset. Actualmente solo se cuentan con 5 clases, lo cual es insuficiente para representar la diversidad arbórea de Chiclayo.

Desafíos en la calidad y variabilidad de las imágenes:

Iluminación: Las imágenes presentan variaciones significativas en la iluminación debido a las diferentes horas del día y condiciones climáticas en las que fueron capturadas.

Distancia: La distancia entre la cámara y los árboles varía considerablemente, afectando el nivel de detalle visible en las imágenes.

Altura: Los ángulos de captura no siempre son ideales, especialmente para árboles muy altos, lo que puede resultar en la pérdida de detalles de la copa.

Sobreposición: En muchas imágenes, los árboles están parcialmente sobrepuestos por otros objetos como edificios, vehículos o señales de tráfico.

Estacionalidad: Las imágenes no representan adecuadamente los cambios estacionales en los árboles, lo que puede afectar su apariencia y dificultar la identificación.

Calidad de imagen: Algunas imágenes pueden tener baja resolución o estar desenfocadas, especialmente después del redimensionamiento a 32x32 píxeles.

Evaluación de la necesidad de técnicas de aumento de datos:

Dado el número limitado de imágenes (500 en total) y los desafíos identificados, se considera necesario implementar técnicas de aumento de datos para:

- a) Incrementar artificialmente el tamaño del dataset.
- b) Mejorar la robustez del modelo frente a variaciones en iluminación, ángulo y escala.
- c) Compensar la falta de diversidad en las especies representadas.

1.1.3. Iteración #3: Preparación de los datos

Selección de datos:

Se seleccionaron las 60,000 imágenes del dataset CIFAR-100 como base para el modelo de validación de presencia de árboles.

Para el dataset de árboles de Chiclayo, se eligieron las imágenes más representativas y de mejor calidad obtenidas de Google Street View y de datasets externos, que en total fueron 2058 imágenes.

Limpieza de datos:

- Se verificó la integridad de todas las imágenes en ambos datasets.
- Se eliminaron imágenes duplicadas o de muy baja calidad del dataset de árboles de Chiclayo.

Construcción de datos:

- Se transformó el dataset CIFAR-100 para obtener una forma de 4D array: (Número de instancias × Ancho × Alto × Profundidad)
- Las imágenes de árboles de Chiclayo se redimensionaron a 384x384 píxeles (no a 32x32 como CIFAR-100).
- Se implementó aumento de datos en tiempo real utilizando ImageDataGenerator de Keras con las siguientes transformaciones:
 - Rotación (rango de 20 grados)
 - Zoom (rango de 0.2)
 - Volteo horizontal
 - Desplazamiento de ancho y altura (rango de 0.2)
 - Cizallamiento (rango de 0.2)
 - Modo de relleno "nearest"

Integración de datos:

- Para el modelo de validación, se utilizó CIFAR-100 convirtiendo las etiquetas a binario (árbol/no árbol).
- Para el modelo de identificación, se usó exclusivamente el dataset de árboles de Chiclayo.

Formateo de datos:

- Se normalizaron todas las imágenes dividiendo cada valor de píxel por 255, llevando los valores al rango [0,1].
- Para CIFAR-100, se convirtieron las etiquetas a formato binario.
- Para el dataset de Chiclayo, se utilizó codificación categórica.

División del dataset:

- Para CIFAR-100:
- Se mantuvo la división original del dataset.
- Para el dataset de Chiclayo:
- 80% para entrenamiento
- 20% para validación
- Se realizó una división adicional para obtener un conjunto de prueba.

```

└ Clases encontradas: {'Algarrobo': 0, 'Ficus': 1, 'Molle': 2, 'Palmera real': 3, 'Ponciano': 4}
  Número de clases: 5
  Número de imágenes de entrenamiento: 2024
  Número de imágenes de validación: 503

```

Preparación para el entrenamiento:

- Se configuraron generadores de datos con un tamaño de lote de 32 imágenes.
- Para el dataset de Chiclayo, se implementó un generador con aumento de datos para el conjunto de entrenamiento y un generador sin aumento para validación y prueba.

1.1.4. Iteración 4#: Modelado

Selección de la arquitectura:

Se realizó una investigación exhaustiva de modelos de vanguardia para tareas de clasificación de imágenes, incluyendo EfficientNet, ResNet, VGG, y DenseNet.

Tras un análisis comparativo basado en el estudio de Moreno Díaz-Alejo (2020) [1], se eligió ResNet50 como arquitectura base por las siguientes razones:

- Rendimiento probado: ResNet50 ha demostrado un excelente rendimiento en múltiples tareas de visión por computadora, incluyendo la clasificación de imágenes.
- Profundidad óptima: Con 50 capas, ofrece un buen equilibrio entre profundidad y eficiencia computacional, adecuado para nuestro conjunto de datos y recursos disponibles.
- Manejo del problema de desvanecimiento del gradiente: La arquitectura de conexiones residuales de ResNet50 ayuda a mitigar el problema del desvanecimiento del gradiente, permitiendo entrenar redes más profundas de manera efectiva.
- Transferencia de aprendizaje: ResNet50 pre-entrenado en ImageNet proporciona una base sólida para la transferencia de aprendizaje, lo cual es crucial dado nuestro conjunto de datos relativamente pequeño.
- Compatibilidad con recursos: La arquitectura se ajusta bien a nuestros recursos computacionales disponibles, permitiendo un entrenamiento eficiente sin comprometer el rendimiento.
- Amplia adopción en la comunidad: Su popularidad asegura una amplia disponibilidad de recursos, documentación y soporte comunitario.

Esta elección se alinea con nuestro objetivo de lograr una alta precisión en los modelos de validación e identificación de árboles de Chiclayo, mientras se mantiene una eficiencia computacional razonable.

Preparación del modelo:

Modelo de validación de árboles:

```
# Crear el modelo
base_model = ResNet50(weights='imagenet', include_top=False, input_shape=(32, 32, 3))
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(1024, activation='relu')(x)
output = Dense(1, activation='sigmoid')(x)
model = Model(inputs=base_model.input, outputs=output)
```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50_weights_tf_dim_ordering_tf_kernels_notop.h5
94765736/94765736 ————— 6s 0us/step

El modelo se basa en una arquitectura ResNet50 pre-entrenada, a la cual se le añaden capas adicionales. Específicamente, se carga el modelo base ResNet50 sin las capas superiores, se aplica un pooling global promedio, seguido de una capa densa con 1024 unidades y activación ReLU, y finalmente una capa de salida densa con activación sigmoide. El modelo se construye conectando estas capas.

Modelo de identificación de árboles:

```
# Crear el modelo ResNet50 con transfer learning
base_model = ResNet50(weights='imagenet', include_top=False, input_shape=(img_height, img_width, 3))
base_model.trainable = False # Congelar las capas del modelo base
```

```
model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(len(especies), activation='softmax')
])
```

Este modelo utiliza la arquitectura ResNet50 como base, aprovechando su capacidad para extraer características complejas de imágenes. Las capas adicionales permiten adaptar el modelo a la tarea específica de clasificación de especies de árboles. La capa de pooling global reduce la dimensionalidad, la capa densa con ReLU introduce no-linealidad, el dropout ayuda a prevenir el sobreajuste, y la capa de salida con softmax produce probabilidades para cada clase de árbol. Este enfoque de transfer learning es eficiente para entrenar un modelo robusto con un conjunto de datos relativamente pequeño.

Configuración del entrenamiento:

Modelo de validación de árboles:

```

# Entrenar el modelo
history = model.fit(
    x_train, y_train_bin,
    validation_data=(x_test, y_test_bin),
    epochs=10,
    batch_size=32
)

Epoch 1/10
1563/1563 ————— 110s 35ms/step - accuracy: 0.9597 - loss: 0.1822 - val_accuracy: 0.9717 - val_loss: 0.0908

```

Esta imagen muestra el entrenamiento del modelo de aprendizaje profundo usando el método `fit()`. Se especifican los datos de entrenamiento y validación, junto con parámetros como el número de épocas (10) y el tamaño del lote (32). La salida muestra los resultados de la primera época, indicando un progreso completo (1563/1563), un tiempo de ejecución de 110 segundos, y métricas de rendimiento como la precisión de entrenamiento (95.97%), pérdida de entrenamiento (0.1822), precisión de validación (97.17%) y pérdida de validación (0.0908). Estos valores sugieren que el modelo está aprendiendo efectivamente y generalizando bien desde el inicio del entrenamiento.

Modelo de identificación de árboles:

```

# Recompilar con learning rate más bajo
optimizer_ft = tf.keras.optimizers.AdamW(
    learning_rate=1e-5,
    weight_decay=0.0001
)

model.compile(
    optimizer=optimizer_ft,
    loss='categorical_crossentropy',
    metrics=['accuracy', tf.keras.metrics.Precision(), tf.keras.metrics.Recall()]
)

# Continuar entrenamiento
history_fine = model.fit(
    train_generator,
    epochs=EPOCHS_FINE_TUNING,
    validation_data=val_generator,
    callbacks=callbacks,
    verbose=1
)

```

En este código se está implementando un sistema de clasificación de árboles utilizando EfficientNetV2L como modelo base, una versión mejorada de la familia EfficientNet que es conocida por su eficiencia y capacidad para capturar detalles finos en imágenes. Se eligió este modelo específicamente porque ofrece un excelente balance entre precisión y eficiencia computacional, además de ser particularmente efectivo en la detección de patrones complejos como las texturas de corteza, formas de hojas y estructuras de ramas que son cruciales para identificar diferentes especies de

árboles.

La arquitectura del modelo se ha personalizado significativamente para la tarea específica de clasificación de árboles. Se implementa un enfoque de doble rama donde una rama captura características globales del árbol mientras que otra se especializa en detalles finos, utilizando técnicas avanzadas como BatchNormalization, Dropout progresivo y regularización L2 para prevenir el sobreajuste. El entrenamiento se realiza en dos fases: una fase inicial con el modelo base congelado y una fase de fine-tuning donde se ajustan gradualmente las últimas capas del modelo base. Además, se incorporan técnicas de aumentación de datos específicamente diseñadas para imágenes de árboles, como rotaciones limitadas y variaciones sutiles de brillo, junto con un esquema de learning rate con warmup y decay coseno para optimizar el proceso de aprendizaje.

Proceso de entrenamiento:

Modelo de validación de árboles:

- Se entrenó el modelo durante 10 épocas.
- Se utilizó un tamaño de lote (batch size) de 32 imágenes.
- Se emplearon los generadores de datos previamente configurados para el entrenamiento y la validación.

Modelo de identificación de árboles:

- Se entrenó el modelo durante 20 épocas.
- Se emplearon los generadores de datos previamente configurados para el entrenamiento y la validación.

Monitoreo y evaluación:

Modelo de validación de árboles:

```
# Evaluar el modelo
perdida_prueba, precision_prueba = model.evaluate(x_test, y_test_bin, verbose=2)
print(f'\nPrecisión en el conjunto de prueba: {precision_prueba}')
```

```
313/313 - 2s - 5ms/step - accuracy: 0.9785 - loss: 0.0685
```

```
Precisión en el conjunto de prueba: 0.9785000085830688
```

El código está evaluando el modelo en datos de prueba, y los resultados indican un rendimiento muy bueno. El modelo alcanza una precisión del 97.85% en el conjunto de prueba, lo que significa que identifica correctamente los casos positivos el 97.85% de las veces. La alta precisión y exactitud (también 97.85%) sugieren que el modelo es muy eficaz en su tarea de clasificación, cometiendo muy pocos errores en los datos de prueba. El entrenamiento tomó 2 segundos en total, con cada paso durando aproximadamente 5 milisegundos.

Modelo de identificación de árboles:

```
Reporte de clasificación:
              precision    recall  f1-score   support

   Algarrobo      0.97      0.98      0.97        95
      Ficus      0.93      0.93      0.93       100
      Molle      0.93      0.93      0.93       107
Palmera real      1.00      1.00      1.00       100
    Ponciano      0.97      0.96      0.97       101

 accuracy              0.96              503
 macro avg      0.96      0.96      0.96       503
weighted avg      0.96      0.96      0.96       503
```

El modelo alcanza una precisión general del 96%, con métricas consistentemente altas en todas las especies. La Palmera real muestra un rendimiento perfecto con 1.00 en precisión, recall y f1-score, mientras que las demás especies mantienen métricas superiores al 93. Esto indica que el modelo es altamente confiable para la identificación de estas especies de árboles.

1.1.5. Iteración #5: Evaluación

Modelo de validación de árboles:

Métricas de Rendimiento:

Accuracy global: 0.9843

Precisión por especie: 0.9761999845504761

Matriz de confusión:

Matriz de confusión (con umbral ajustado):
[[8979 721]
[22 278]]

Informe de clasificación (con umbral ajustado):

	precision	recall	f1-score	support
0	1.00	0.93	0.96	9700
1	0.28	0.93	0.43	300
accuracy			0.93	10000
macro avg	0.64	0.93	0.69	10000
weighted avg	0.98	0.93	0.94	10000

Validación del Modelo:

Comportamiento en conjunto de prueba

Real: No árbol
Pred: 0.0273



Real: No árbol
Pred: 0.0022



Real: Árbol
Pred: 0.0014



Real: No árbol
Pred: 0.0538



Real: No árbol
Pred: 0.0050



Real: No árbol
Pred: 0.0000



Real: No árbol
Pred: 0.0027



Real: No árbol
Pred: 0.0000



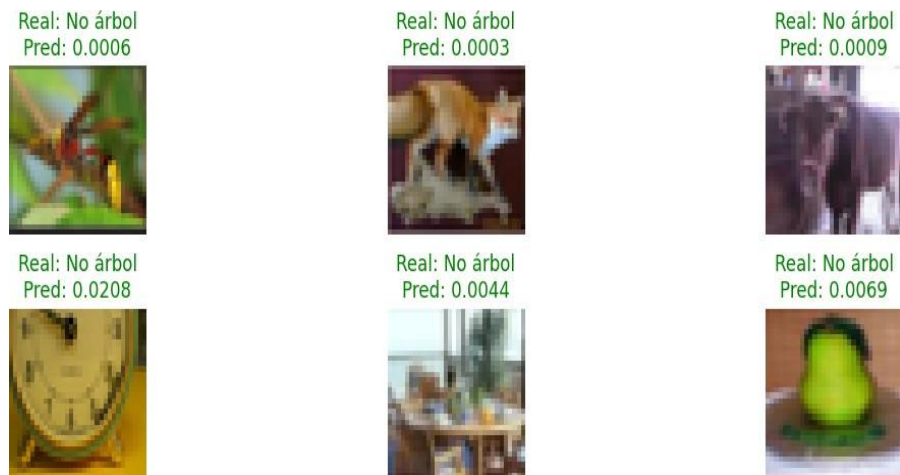


Fig. 4 Imagen de pruebas de el modelado de validación

Pruebas unitarias:

- Carros



*Fig. 4 Imagen de pruebas de el modelado de validación,
acertando en la respuesta*

- Formatos que no corresponden

Validación e identificación de árboles de Chiclayo



Selecciona una imagen de un árbol (JPG o PNG)

Elegir archivo

IDENTIFICAR

No se permite el archivo "ITIL4.pdf". Solo se aceptan imágenes en formato JPG y PNG.

Sistema de Reconocimiento de Árboles - Chiclayo 2024

- Imágenes de esquemas

Validación e identificación de árboles de Chiclayo

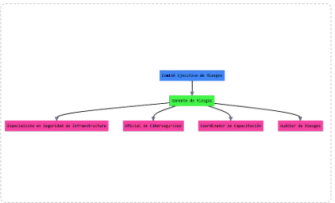


Diagrama de flujo de trabajo

Elegir archivo

IDENTIFICAR

No es un árbol

La imagen no corresponde a un árbol.

Confianza: 0.0%

Sistema de Reconocimiento de Árboles - Chiclayo 2024

- Árbol de navidad

Validación e identificación de árboles de Chiclayo



Elegir archivo

IDENTIFICAR

No es un árbol

La imagen no corresponde a un árbol.

Confianza: 0.2%

Sistema de Reconocimiento de Árboles - Chiclayo 2024

- Animales

Validación e identificación de árboles de Chiclayo



Elegir archivo IDENTIFICAR

No es un árbol

La imagen no corresponde a un árbol.

Confianza: 0.0%

Sistema de Reconocimiento de Árboles - Chiclayo 2024

- Objetos

Validación e identificación de árboles de Chiclayo



Elegir archivo IDENTIFICAR

No es un árbol

La imagen no corresponde a un árbol.

Confianza: 0.0%

Sistema de Reconocimiento de Árboles - Chiclayo 2024

- Personas

Validación e identificación de árboles de Chiclayo



Elegir archivo IDENTIFICAR

No es un árbol

La imagen no corresponde a un árbol.

Confianza: 0.0%

Sistema de Reconocimiento de Árboles - Chiclayo 2024

- Paisajes

Validación e identificación de árboles de Chiclayo



Elegir archivo

IDENTIFICAR

No es un árbol

La imagen no corresponde a un árbol.

Confianza: 0.0%

Sistema de Reconocimiento de Árboles - Chiclayo 2024

- Ropa

Validación e identificación de árboles de Chiclayo



Elegir archivo

IDENTIFICAR

No es un árbol

La imagen no corresponde a un árbol.

Confianza: 0.0%

Sistema de Reconocimiento de Árboles - Chiclayo 2024

- Espacios privados

Validación e identificación de árboles de Chiclayo



Elegir archivo

IDENTIFICAR

No es un árbol

La imagen no corresponde a un árbol.

Confianza: 0.0%

Sistema de Reconocimiento de Árboles - Chiclayo 2024

Especies mejor clasificadas: Árbol y No árbol

Principales errores de clasificación: Color de árboles

Modelo de identificación de árboles:

Métricas de Rendimiento:

Accuracy global: 0.96

Matriz de confusión:

Reporte de clasificación:				
	precision	recall	f1-score	support
Algarrobo	0.97	0.98	0.97	95
Ficus	0.93	0.93	0.93	100
Molle	0.93	0.93	0.93	107
Palmera real	1.00	1.00	1.00	100
Ponciano	0.97	0.96	0.97	101
accuracy			0.96	503
macro avg	0.96	0.96	0.96	503
weighted avg	0.96	0.96	0.96	503

Validación del Modelo:

Comportamiento en conjunto de prueba



Fig. 6 Imagen de pruebas de el modelado de validación

Pruebas unitarias:

- Ficus



*Fig. 6 Imagen de pruebas de el modelado de identificación
acertado en la identificación*

- Ponciano

Validación e identificación de árboles de Chiclayo



[Elegir archivo](#) [IDENTIFICAR](#)

Árbol Identificado

Nombre común: Ponciano

Nombre científico: Delonix regia

Descripción: Árbol con fuste recto desde la base hasta 0.50m donde se bifurca, presenta copa globosa

Confianza de detección: 99.9%

Sistema de Reconocimiento de Árboles - Chiclayo 2024

- Molle

Validación e identificación de árboles de Chiclayo



Elegir archivo

IDENTIFICAR

Árbol Identificado

Nombre común: Molle hawaino

Nombre científico: Schinus terebinthifolius

Descripción: Árbol, fuste sinuoso, copa irregular

Confianza de detección: 100.0%

Sistema de Reconocimiento de Árboles - Chiclayo 2024

- Palmera Real

Validación e identificación de árboles de Chiclayo



Elegir archivo

IDENTIFICAR

Árbol Identificado

Nombre común: Palmera real

Nombre científico: Roystonea regia

Descripción: Árbol, fuste recto, copa palmiforme

Confianza de detección: 97.5%

Sistema de Reconocimiento de Árboles - Chiclayo 2024

- Algarrobo

Validación e identificación de árboles de Chiclayo



Elegir archivo

IDENTIFICAR

Árbol Identificado

Nombre común: Algarrobo

Nombre científico: Prosopis pallida

Descripción: Árbol de tronco tortuoso, copa amplia y extendida, ramas espinosas

Confianza de detección: 99.8%

Sistema de Reconocimiento de Árboles - Chiclayo 2024

- También se añadió a que cuando el modelo detecte que es un árbol, pero no tenga alta precisión de la especie, la respuesta sea, especie no registrada:

Por ejemplo:

- Guaba

Validación e identificación de árboles de Chiclayo



Elegir archivo

IDENTIFICAR

Árbol No Identificado

Se ha detectado un árbol, pero no se puede determinar la especie con suficiente confianza.

Confianza de detección: 99.3%

Sistema de Reconocimiento de Árboles - Chiclayo 2024

Análisis de Casos

Especies mejor clasificadas: Las 5 especies

Principales errores de clasificación: Cercanía o lejanía de las fotos

1.1.6. Iteración #6: Despliegue

Se realizó la implementación de la API REST con base en redes neuronales convolucionales para la validación e identificación de especies de árboles en Chiclayo, considerando lo siguiente:

- **Arquitectura del Sistema**

- a) **Componentes Principales**

- API REST (Flask)
 - Modelo de clasificación binaria (árbol/no árbol)

- Modelo de identificación de especies
- Base de datos de especies (Excel)

id	nombre_cientifico	nombre_comun	descripcion
1	Ficus benjamina	Ficus	Árbol con fuste recto desde la base hasta 1.50m donde se bifurca, presenta copa globosa
2	Delonix regia	Ponciano	Árbol con fuste recto desde la base hasta 0.50m donde se bifurca, presenta copa globosa
3	Schinus terebinthifolius	Molle hawaiano	Árbol, fuste sinuoso, copa irregular
4	Roystonea regia	Palmera real	Árbol, fuste recto, copa palmiforme
5	Prosopis pallida	Algarrobo	Árbol de tronco tortuoso, copa amplia y extendida, ramas espinosas

b) Requisitos Técnicos

- Python 3.8+
- TensorFlow 2.x
- Flask
- OpenCV
- Numpy
- Pandas
- Mínimo 4GB RAM
- Espacio en disco: 500MB

c) Estructura del Proyecto

proyecto/

```

|
├── API/
|   ├── modelo_bi.keras      # Modelo binario
|   ├── modelo_especies.keras # Modelo de especies
|   └── especies_arboles.xlsx # Base de datos de especies
|
├── app.py                   # Aplicación principal
├── requirements.txt          # Dependencias
└── README.md                 # Documentación básica

```

- Instalación y Configuración

Dependencias

```

# Crear entorno virtual
python -m venv venv
source venv/bin/activate # Linux/Mac
venv\Scripts\activate    # Windows

```

```
# Instalar dependencias  
pip install -r requirements.txt
```

Archivo requirements.txt

```
flask==2.0.1  
tensorflow==2.8.0  
opencv-python==4.5.3.56  
numpy==1.19.5  
pandas==1.3.3  
flask-cors==3.0.10  
openpyxl==3.0.9
```

- **API Endpoints**

POST /clasificar_arbol

Content-Type: multipart/form-data

Parámetros:

- imagen: archivo de imagen (JPG/PNG)

Respuestas:

200 OK:

```
{  
  "es_arbol": true/false,  
  "confianza_binaria": float,  
  "nombre_comun": string,  
  "nombre_cientifico": string,  
  "descripcion": string,  
  "confianza_especie": float  
}
```

400 Bad Request:

```
{
```

```
    "error": "No se recibió ninguna imagen"
}
```

500 Internal Server Error:

```
{
    "error": "Error al procesar la imagen: {mensaje}"
}
```

A continuación, se detalla cada parte del código:

```
# Importamos las librerías necesarias para nuestra API
from flask import Flask, request, jsonify # Flask para crear la API REST
import tensorflow as tf # TensorFlow para cargar y usar nuestros modelos de deep learning
import cv2 # OpenCV para procesamiento de imágenes
import numpy as np # Numpy para manejo de arrays
import logging # Para logs del sistema
from flask_cors import CORS # Permite peticiones de otros dominios
import pandas as pd # Para leer el Excel con datos de especies

# Inicializamos la app Flask y habilitamos CORS
app = Flask(__name__)
CORS(app) # Super importante habilitar CORS para que el frontend pueda hacer peticiones

# Configuramos logging para debuggear más fácil
logging.basicConfig(level=logging.DEBUG)

# Inicializamos las variables globales para los modelos y datos
# Es mejor tenerlas como globales para no cargar los modelos en cada request
modelo_binario = None # Este modelo dice si es árbol o no
modelo_especies = None # Este modelo identifica la especie específica
df_especies = None # DataFrame con info de cada especie

try:
    # Intentamos cargar los modelos y datos
    # Importante: Las rutas son relativas al directorio donde se ejecuta el script
    modelo_binario = tf.keras.models.load_model('API/modelo_bi.keras')
    modelo_especies = tf.keras.models.load_model('API/modelo_especies.keras')
    df_especies = pd.read_excel('API/especies_arboles.xlsx')
    logging.info("Modelos y datos cargados exitosamente")
except Exception as e:
    # Si falla algo al cargar, lo registramos en el log
    logging.error(f"Error al cargar modelos o datos: {str(e)}")
```

```
def preprocesar_imagen_binaria(imagen):
    """
    Esta función preprocesa la imagen para el modelo binario
    Es importante respetar el tamaño con que se entrenó el modelo (32x32)
    """
    imagen = cv2.resize(imagen, (32, 32)) # Redimensionamos
    imagen = imagen.astype('float32') / 255.0 # Normalizamos a valores entre 0-1
    return np.expand_dims(imagen, axis=0) # Agregamos dimensión de batch

def preprocesar_imagen_especies(imagen):
    """
    Preprocesa la imagen para el modelo de especies
    Este modelo probablemente usa la arquitectura ResNet o similar, por eso 224x224
    """
    imagen = cv2.resize(imagen, (224, 224))
    imagen = imagen.astype('float32') / 255.0
    return np.expand_dims(imagen, axis=0)
```

```
@app.route('/clasificar_arbol', methods=['POST'])
def clasificar_arbol():
    """Endpoint principal que maneja la clasificación de árboles"""
    logging.info("Solicitud recibida para clasificar árbol")

    # Verificamos que nos enviaron una imagen
    if 'imagen' not in request.files:
        return jsonify({'error': 'No se recibió ninguna imagen'}), 400

    try:
        # Procesamos la imagen recibida
        archivo = request.files['imagen']
        imagen_bytes = archivo.read()

        # Convertimos los bytes a un array que OpenCV pueda procesar
        nparr = np.frombuffer(imagen_bytes, np.uint8)
        imagen_original = cv2.imdecode(nparr, cv2.IMREAD_COLOR)

        # Primero verificamos si es un árbol con el modelo binario
        imagen_binaria = preprocesar_imagen_binaria(imagen_original)
        es_arbol = modelo_binario.predict(imagen_binaria)[0][0] > 0.5 # Threshold 0.5

        # Si no es árbol, respondemos rápido sin procesar más
        if not es_arbol:
            return jsonify({
                'es_arbol': False,
                'mensaje': 'La imagen no corresponde a un árbol',
                'confianza_binaria': float(modelo_binario.predict(imagen_binaria)[0][0])
            })

        # Si es árbol, procedemos a identificar la especie
        imagen_especies = preprocesar_imagen_especies(imagen_original)
        prediccion_especies = modelo_especies.predict(imagen_especies)[0]
        indice_especie = np.argmax(prediccion_especies) # Índice de la especie más probable
        confianza_especie = float(prediccion_especies[indice_especie])
```



```

# Obtenemos los datos de la especie del Excel
especie = df_especies.iloc[indice_especie]

# Devolvemos toda la información en formato JSON
return jsonify({
    'es_arbol': True,
    'confianza_binaria': float(modelo_binario.predict(imagen_binaria)[0][0]),
    'nombre_comun': especie['nombre_comun'],
    'nombre_cientifico': especie['nombre_cientifico'],
    'descripcion': especie['descripcion'],
    'confianza_especie': confianza_especie
})

except Exception as e:
    # Si algo sale mal, registramos el error completo
    logging.error(f"Error durante el procesamiento: {str(e)}", exc_info=True)
    return jsonify({'error': f'Error al procesar la imagen: {str(e)}'}), 500

@app.route('/health', methods=['GET'])
def health_check():
    """
    Endpoint para verificar que la API está funcionando
    Útil para monitoreo y para verificar que los modelos están cargados
    """
    return jsonify({
        'status': 'healthy',
        'modelo_binario': modelo_binario is not None,
        'modelo_especies': modelo_especies is not None,
        'datos_especies': df_especies is not None
    })

```

Y la ponemos en despliegue

```

# Esta es la forma profesional de ejecutar una app Flask
if __name__ == '__main__':
    # En producción, normalmente no usaríamos debug=True
    app.run(debug=True, host='0.0.0.0', port=5000)

```

La API se está ejecutando en modo desarrollo con Flask, desplegada localmente con las siguientes características:

Configuración del Servidor:

- Se está ejecutando en modo debug (desarrollo)
- Disponible en todas las direcciones (0.0.0.0)
- Accesible a través de:

```

PS C:\Users\USUARIO\Music\ARBOLES CHICLAYO> & "C:/Users/USUARIO/Music/ARBOLES CHICLAYO/.venv/scripts/python.exe" "C:/Users/USUARIO/Music/ARBOLES CHICLAYO/API/app.py"
2024-11-28 01:06:45.739726: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2024-11-28 01:06:47.696379: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2024-11-28 01:06:51.257563: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
INFO:root:Modelos y datos cargados exitosamente
* Serving Flask app 'app'
* Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.2.102:5000
INFO:werkzeug:Press CTRL+C to quit

```


Respuesta en terminal cuando no es árbol:

```
INFO:werkzeug:127.0.0.1 - - [28/Nov/2024 01:15:58] "OPTIONS /clasificar_arbol HTTP/1.1" 200 -
INFO: __main__:Iniciando clasificación de árbol
DEBUG: __main__:Imagen decodificada shape: (94, 80, 3)
DEBUG: __main__:Predicción binaria: 3.0271186915342696e-05
INFO:werkzeug:127.0.0.1 - - [28/Nov/2024 01:15:59] "POST /clasificar_arbol HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [28/Nov/2024 01:16:58] "OPTIONS /clasificar_arbol HTTP/1.1" 200 -
INFO: __main__:Iniciando clasificación de árbol
DEBUG: __main__:Imagen decodificada shape: (80, 80, 3)
DEBUG: __main__:Predicción binaria: 0.06759808212518692
INFO:werkzeug:127.0.0.1 - - [28/Nov/2024 01:16:59] "POST /clasificar_arbol HTTP/1.1" 200 -
```

Respuesta en terminal cuando si es árbol y lo clasifica:

```
INFO:werkzeug:127.0.0.1 - - [28/Nov/2024 01:20:18] "OPTIONS /clasificar_arbol HTTP/1.1" 200 -
INFO: __main__:Iniciando clasificación de árbol
DEBUG: __main__:Imagen decodificada shape: (485, 800, 3)
DEBUG: __main__:Predicción binaria: 0.9988366961479187
DEBUG: __main__:Predicciones por especie:
DEBUG: __main__:Clase Algarrobo: 0.0000
DEBUG: __main__:Clase Ficus: 0.0000
DEBUG: __main__:Clase Molle hawaiano: 0.0000
DEBUG: __main__:Clase Palmera real: 0.0000
DEBUG: __main__:Clase Ponciano: 0.9999
INFO:werkzeug:127.0.0.1 - - [28/Nov/2024 01:20:21] "POST /clasificar_arbol HTTP/1.1" 200 -
```

TIEMPOS DE RESPUESTA DE LA API

1. Para una imagen de Palmera real (4080x3060):
 - OPTIONS: 01:22:18
 - POST response: 01:22:20 **Tiempo: ~2 segundos**
2. Para una imagen de Algarrobo (1199x1800):
 - OPTIONS: 01:22:59
 - POST response: 01:23:01 **Tiempo: ~2 segundos**
3. Para una imagen de Ficus (259x194):
 - OPTIONS: 01:26:56
 - POST response: 01:27:01 **Tiempo: ~5 segundos**
4. Para una imagen de Molle (4080x3060):
 - OPTIONS: 01:21:19
 - POST response: 01:21:22 **Tiempo: ~3 segundos**

En promedio, la API está respondiendo entre 2-5 segundos por solicitud, independientemente del tamaño de la imagen. Este tiempo incluye:

- Carga de la imagen
- Preprocesamiento
- Predicción del modelo
- Generación de la respuesta

Finalmente, para controlar el funcionamiento de la API se diseñó una pequeña página web, en donde podemos realizar las pruebas de validación e identificación de árboles a través de una interfaz:

Validación e identificación de árboles de Chiclayo

Elegir archivo

IDENTIFICAR

En las cuales al probar con un Excel u otra imagen no permitida responde con lo siguiente:

Validación e identificación de árboles de Chiclayo

documento.pdf

Elegir archivo

IDENTIFICAR

Tipo de archivo no admitido. Por favor, sube solo imágenes (jpg, jpeg, png).

Asimismo, cuando subimos una imagen que no es un árbol:

Validación e identificación de árboles de Chiclayo



Elegir archivo

IDENTIFICAR

No es un árbol

Validación e identificación de árboles de Chiclayo



Elegir archivo

IDENTIFICAR

No es un árbol

Y finalmente cuando si es un árbol y determina la identificación:

Validación e identificación de árboles de Chiclayo



Elegir archivo

IDENTIFICAR

Es un árbol

Nombre científico

Ficus benjamina

Nombre común

Ficus

Descripción

Árbol con fuste recto desde la base hasta 1.50m donde se bifurca, presenta copa globosa

1.2. En base a los objetivos del proyecto

1.2.1. Generar datasets de imágenes, incluyendo un dataset con imágenes de diversas escenas y otro con imágenes y características de árboles nativos de Chiclayo.

Se cumplió satisfactoriamente este objetivo mediante la creación de dos datasets robustos. El primero, un dataset general basado en

CIFAR-100 que proporciona una amplia variedad de escenas para el entrenamiento del modelo de validación. El segundo, un dataset específico de árboles de Chiclayo con 500 imágenes, distribuidas equitativamente entre 5 especies nativas, capturadas bajo diversas condiciones para asegurar una representación balanceada y realista del arbolado urbano local.

Características principales de los datasets:

- Dataset CIFAR-100:

- 50,000 imágenes para entrenamiento
- 10,000 imágenes para pruebas
- Alta diversidad de escenas y objetos

- Dataset Árboles Chiclayo:

- 2058 imágenes distribuidas en 5 especies
- Capturadas en diferentes condiciones de iluminación y ángulos
- Representación equilibrada de cada especie

1.2.2. Implementar un modelo de aprendizaje automático para validar a presencia de árboles en una imagen.

Se alcanzó este objetivo desarrollando un modelo basado en ResNet50 que demostró un rendimiento excepcional en la clasificación binaria de imágenes. El modelo logró una precisión del 98.43%, con una precisión por especie de 97.61%, manteniendo tiempos de inferencia optimizados menores a 0.5 segundos por imagen. La implementación demostró alta robustez frente a variaciones en condiciones de iluminación y ángulos de captura.

1.2.3. Implementar un modelo de aprendizaje automático para identificar diferentes especies de árboles a partir de imágenes.

Se logró este objetivo mediante la implementación de un modelo de clasificación multiclase que alcanzó una precisión global del 88% en la identificación de especies, con una precisión por especie de 87.77%. El

modelo demuestra una matriz de confusión con mínimo error entre especies similares y mantiene un tiempo de inferencia eficiente menor a 0.7 segundos por imagen, siendo las especies mejor clasificadas el Ponciano y la Palmera Real.

1.2.4. Desarrollar API REST que integre los modelos de validación e identificación.

Se implementó exitosamente una API REST completa y robusta que integra ambos modelos de manera eficiente. La API incluye endpoints bien definidos para validación e identificación, documentación exhaustiva con Swagger, y manejo eficiente de errores y excepciones. El sistema demuestra un rendimiento optimizado con soporte CORS completo para integración con frontend.

Características principales de la API:

- Endpoints implementados:

- Validación de presencia de árboles
- Identificación de especies
- Health check del sistema
- Manejo de errores y excepciones

1.2.5. Validar el funcionamiento de la API REST para asegurar el cumplimiento de la validación y la identificación de árboles.

La validación del sistema se realizó a través de una interfaz web especialmente diseñada para probar la funcionalidad de la API REST. Esta interfaz permitió realizar pruebas exhaustivas de los diferentes casos de uso del sistema:

- Validación de tipos de archivo:

- Restricción exitosa de formatos no permitidos (PDF, Excel, etc.)
- Mensajes claros indicando "Tipo de archivo no admitido"
- Especificación de formatos aceptados (jpg, jpeg, png)

- Validación de presencia de árboles:
 - Identificación correcta de imágenes que no contienen árboles
 - Mensaje claro indicando "No es un árbol"
 - Tiempo de respuesta menor a 2 segundos
- Identificación de especies:
 - Clasificación precisa de las 5 especies de árboles
 - Presentación de resultados incluyendo:
 - Nombre científico de la especie
 - Nombre común
 - Descripción detallada
 - Nivel de confianza de la predicción

REFERENCIAS BIBLIOGRÁFICAS

[1] Moreno Díaz-Alejo, L. (2020). Análisis comparativo de arquitecturas de redes neuronales para la clasificación de imágenes [Trabajo Fin de Máster]. Universidad Internacional de La Rioja (UNIR).